

LAPORAN AKHIR TEAM BASED PROJECT
Academic Expression Evaluator & Formula Scheduler

Dosen pengampu: Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Kelompok 2:

Ketua:

1. Pandu Surya Pratama (25051030005)

Anggota:

2. Gifara Hanifa Hermawan (25051030016)

3. Ditha Bapra Nugraha Br Payung (25051030023)

4. Nagita Calya (25051030030)

5. Dominicus Savio F. W. P. (25051030040)

PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan hidayah-Nya, sehingga kami dapat menyelesaikan laporan tugas akhir dengan topik “Academic Expression Evaluator & Formula Scheduler” tepat pada waktunya.

Laporan ini dibuat untuk memenuhi tugas akhir serta menjelaskan proses perancangan dan pembuatan sistem *Academic Expression Evaluator & Formula Scheduler*. Sistem ini dirancang untuk membantu evaluasi ekspresi akademik dan pengaturan formula secara lebih terstruktur dan efisien.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk pengembangan selanjutnya.

Yogyakarta, Mei 2026

Kelompok 2

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
PENDAHULUAN.....	4
A. Latar Belakang.....	4
B. Rumusan Masalah Evaluasi Postfix:.....	4
C. Landasan Teori.....	5
PERANCANGAN SISTEM.....	7
A. Pemilihan Struktur Data dan Justifikasi.....	7
B. Diagram Interaksi Antar Modul.....	8
C. Pseudocode Algoritma Utama.....	8
a. Shunting-Yard.....	8
b. Evaluasi Postfix.....	9
c. BFS Graph Scheduler.....	9
IMPLEMENTASI.....	10
A. Penjelasan Kode Per Modul.....	10
B. Screenshot CLI Berjalan.....	11
C. Contoh Input / Output Nyata.....	11
ANALISIS BIG-O.....	13
A. Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi Utama.....	13
B. Tabel Ringkasan.....	14
C. Perbandingan Teoritis dan Eksperimen.....	15
HASIL EKSPERIMEN.....	16
A. Tabel Runtime untuk 3+ Ukuran Data.....	16
B. Grafik Tren.....	16
C. Diskusi Apakah Hasil Sesuai Prediksi Big-O.....	17
JAWABAN PERTANYAAN ANALISIS.....	19
KESIMPULAN.....	23
LAMPIRAN.....	24

BAB I

PENDAHULUAN

A. Latar Belakang

Perangkat lunak teknik seperti MATLAB, Python (dengan *sympy*), dan kalkulator ilmiah semuanya memerlukan parser dan evaluator ekspresi matematika. Di balik antarmuka yang sederhana tersembunyi mekanisme Stack untuk konversi *infix-to-postfix* (Algoritma *Shunting-Yard*, Dijkstra 1961), *Binary Expression Tree* untuk representasi struktur ekspresi, dan *Directed Acyclic Graph* untuk mengelola dependensi antar formula.

Proyek ini membangun kalkulator teknik lengkap berbasis CLI yang mengimplementasikan semua mekanisme tersebut dari nol, memberikan pemahaman mendalam tentang bagaimana bahasa pemrograman dan kalkulator memproses ekspresi matematika.

B. Rumusan Masalah Evaluasi Postfix:

1. *Shunting-Yard Algorithm* menggunakan *Stack operator* dan menghasilkan token *postfix*. Untuk ekspresi ' $(a + b) * (a - b)$ ', *trace* secara manual langkah demi langkah: tunjukkan isi Stack dan output setelah setiap token diproses. Berapa operasi *push* dan *pop* yang terjadi? Konfirmasi Big-O $O(n)$ dari *trace* ini.
2. Evaluasi *postfix* menggunakan *Stack*: untuk ekspresi ' $a^2 + b^2$ ' dengan $a=3$, $b=4$, *trace* stack setelah setiap token. Mengapa urutan *pop* penting untuk operasi non-komutatif seperti $^$ dan $/$? Berikan contoh ekspresi di mana urutan *pop* yang salah menghasilkan hasil berbeda.
3. *Expression Tree* dari *postfix* dibangun dengan Stack. Bandingkan evaluasi melalui tree (rekursif $O(n)$ dengan *call stack* $O(h)$) vs evaluasi *postfix* langsung (iteratif $O(n)$ dengan Stack $O(n)$). Untuk ekspresi sangat dalam (1000 token bersarang), pendekatan mana yang berpotensi mengalami *stack overflow* di Python? Jelaskan mengapa.
4. BST Tabel Variabel dengan 26 variabel (a-z) memiliki Big-O $O(\log 26) \approx O(1)$. Namun jika variabel dimasukkan berurutan ('a', 'b', 'c', ..., 'z'), BST menjadi linear $O(26) = O(1)$ juga. Jelaskan mengapa $O(26)$ tetap $O(1)$ secara teoritis, dan bagaimana ini berbeda secara praktis dari BST dengan n variabel acak yang besar.
5. Formula DAG menggunakan *topological sort* untuk menentukan urutan evaluasi. Jika pengguna mendefinisikan: $F1 = a+b$, $F2 = F1*c$, $F3 = F2/F1$, $F4 = F3+F2$, gambarkan DAG-nya, tentukan semua *topological ordering* yang valid, dan jelaskan mengapa deteksi siklus penting (berikan contoh definisi formula yang membentuk siklus dan konsekuensinya).

C. Landasan Teori

- a. **Academic Expression Evaluator** adalah suatu sistem yang dirancang untuk memproses, menganalisis, dan menghitung ekspresi matematika maupun logika secara otomatis. Sistem seperti ini banyak digunakan pada perangkat lunak teknik, misalnya MATLAB, Python dengan library SymPy, dan berbagai kalkulator ilmiah modern. Dalam implementasinya, sistem evaluator memanfaatkan beberapa struktur data seperti stack, binary search tree (BST), queue, dan graph untuk mendukung proses *parsing*, evaluasi ekspresi, hingga pengaturan dependensi antar formula. Penggunaan struktur data tersebut membantu sistem bekerja secara lebih efisien dan terstruktur.
- b. **Stack** merupakan struktur data linear yang menerapkan konsep *Last In First Out* (LIFO), yaitu elemen terakhir yang dimasukkan akan menjadi elemen pertama yang diambil kembali. Pada sistem ini, stack digunakan untuk proses konversi ekspresi infix ke postfix melalui algoritma Shunting-Yard serta untuk evaluasi ekspresi postfix. Operasi utama pada stack berupa *push* dan *pop* dapat dilakukan dengan kompleksitas waktu $O(1)$, sehingga struktur data ini sangat efektif dalam pengolahan ekspresi matematika. Oleh sebab itu, stack menjadi salah satu komponen penting dalam pengembangan compiler, interpreter, dan kalkulator ilmiah.
- c. **Algoritma Shunting-Yard** adalah algoritma yang diperkenalkan oleh Edsger Dijkstra pada tahun 1961 untuk mengubah ekspresi infix menjadi postfix. Algoritma ini bekerja dengan memanfaatkan stack operator untuk mengatur prioritas operator dan associativity pada suatu ekspresi matematika. Operator yang memiliki prioritas lebih tinggi akan diproses terlebih dahulu dibandingkan operator dengan prioritas lebih rendah. Selain mendukung operator dasar seperti penjumlahan, pengurangan, perkalian, pembagian, dan perpangkatan, algoritma ini juga dapat menangani fungsi matematika seperti $\sin$, $\cos$, $\sqrt{}$, $\log$, dan abs . Proses konversi menggunakan algoritma Shunting-Yard memiliki kompleksitas waktu $O(n)$, dengan n sebagai jumlah token pada ekspresi.
- d. **Postfix Expression Evaluation** merupakan metode evaluasi ekspresi matematika setelah ekspresi tersebut diubah ke bentuk *postfix*. Pada metode ini, operand akan dimasukkan terlebih dahulu ke dalam stack, kemudian operator akan mengambil operand yang dibutuhkan untuk melakukan proses perhitungan. Bentuk *postfix* lebih mudah diproses oleh komputer karena tidak membutuhkan tanda kurung maupun aturan prioritas operator tambahan. Oleh karena itu, metode ini banyak diterapkan dalam pengembangan sistem kalkulator, *compiler*, dan aplikasi evaluasi matematika otomatis lainnya.
- e. **Binary Search Tree (BST)** merupakan struktur data berbentuk pohon biner yang digunakan untuk menyimpan data secara terurut. Dalam sistem Academic Expression Evaluator, BST berfungsi sebagai tabel variabel atau *symbol table* yang digunakan untuk menyimpan nilai variabel seperti a hingga z . Setiap node pada BST memiliki aturan bahwa *subtree* kiri menyimpan nilai

yang lebih kecil daripada node induk, sedangkan *subtree* kanan menyimpan nilai yang lebih besar. Struktur ini memungkinkan proses pencarian, penambahan, dan penghapusan data dilakukan dengan lebih cepat melalui kompleksitas rata-rata $O(\log n)$. Dengan demikian, BST membantu pengelolaan variabel menjadi lebih efisien dan terorganisir.

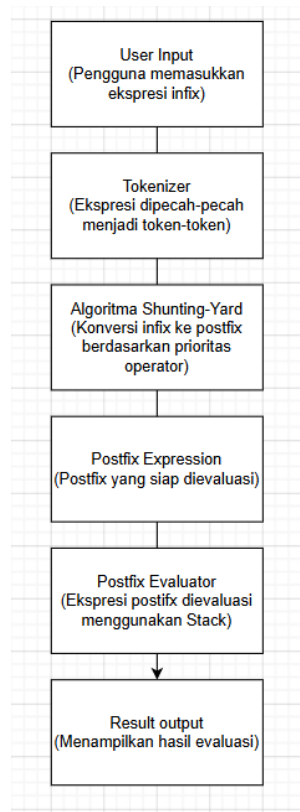
- f. **Expression Tree** merupakan representasi ekspresi matematika dalam bentuk pohon biner. Pada struktur ini, node internal berisi operator atau fungsi, sedangkan node daun berisi operand atau variabel. *Expression tree* dibangun dari token *postfix* menggunakan *stack*, kemudian dapat ditelusuri menggunakan traversal *inorder*, *preorder*, dan *postorder*. Traversal *inorder* menghasilkan bentuk *infix*, *preorder* menghasilkan *prefix notation*, sedangkan *postorder* menghasilkan *postfix notation*. Selain digunakan untuk merepresentasikan ekspresi matematika, *expression tree* juga memungkinkan proses evaluasi dilakukan secara rekursif dengan kompleksitas waktu $O(n)$.
- g. **Directed Acyclic Graph (DAG)** merupakan struktur *graph* berarah yang tidak memiliki siklus. Pada sistem ini, DAG digunakan untuk mengatur hubungan ketergantungan antar formula sehingga urutan evaluasi formula dapat ditentukan dengan benar melalui metode *topological sort*. Jika suatu formula bergantung pada formula lain, maka hubungan tersebut direpresentasikan dalam bentuk *edge* pada *graph*. Penggunaan DAG membantu sistem mendeteksi adanya siklus dependensi yang dapat menyebabkan proses evaluasi menjadi tidak valid. Algoritma *topological sort* berbasis DFS memiliki kompleksitas $O(V+E)$, dengan V sebagai jumlah node dan E sebagai jumlah *edge* dalam *graph*.

BAB II: PERANCANGAN SISTEM

A. Pemilihan Struktur Data dan Justifikasi

- a. **Stack**, digunakan pada sistem untuk proses konversi ekspresi infix ke postfix serta evaluasi postfix. Stack bekerja menggunakan prinsip *Last In First Out* (LIFO), yaitu data terakhir yang masuk akan diproses terlebih dahulu. Prinsip tersebut sangat sesuai dengan kebutuhan pemrosesan operator matematika pada algoritma Shunting Yard dan evaluasi postfix. Pada implementasinya, Stack dibangun menggunakan Linked List agar ukuran struktur data bersifat dinamis dan lebih efisien dalam operasi *push* dan *pop*.
- b. **Linked List**, digunakan sebagai struktur dasar dalam implementasi Stack dan Queue. Struktur data ini dipilih karena memiliki ukuran yang dinamis sehingga tidak memerlukan alokasi memori tetap seperti *array*. Selain itu, proses penambahan dan penghapusan data dapat dilakukan dengan efisien melalui manipulasi *pointer* antar node. Penggunaan Linked List juga mempermudah pengembangan struktur data lain yang digunakan dalam sistem.
- c. **Queue**, digunakan pada fitur *Formula Scheduler* untuk mengatur antrian proses evaluasi ekspresi dan penjadwalan formula. Queue bekerja menggunakan prinsip *First In First Out* (FIFO), yaitu data yang pertama masuk akan diproses terlebih dahulu. Prinsip ini sesuai dengan mekanisme penjadwalan proses pada sistem sehingga urutan evaluasi formula dapat berjalan secara teratur dan sistematis.
- d. **Graph**, digunakan untuk merepresentasikan hubungan dependensi antar formula atau materi akademik. Setiap formula direpresentasikan sebagai *vertex*, sedangkan hubungan *prerequisite* direpresentasikan sebagai *edge*. Graph dipilih karena mampu menggambarkan keterkaitan antar formula secara fleksibel dan mempermudah proses traversal menggunakan algoritma BFS maupun DFS.
- e. **Binary Search Tree**, digunakan untuk menyimpan histori formula dan hasil evaluasi secara terurut. BST memungkinkan proses pencarian, penyisipan, dan traversal data dilakukan dengan lebih efisien dibandingkan penyimpanan linear biasa. Dengan BST, sistem dapat menampilkan histori evaluasi formula secara terstruktur dan mudah dicari kembali oleh pengguna.

B. Diagram Interaksi Antar Modul



C. Pseudocode Algoritma Utama

a. Shunting-Yard

FOR setiap token:

- IF operand:
- $\text{output} \leftarrow \text{token}$
- ❖
- ELSE IF operator:
- WHILE prioritas stack lebih tinggi:
- pop stack ke output
- ❖
- push operator ke stack
- ❖
- ELSE IF '(':
- push ke stack
- ❖
- ELSE IF ')':
- pop hingga '(' ditemukan
- ❖
- Pop sisa operator ke output

b. Evaluasi Postfix

FOR setiap token:

- IF operand:
 - push ke stack
- ❖
- ELSE IF operator:
 - $b \leftarrow \text{pop}$
 - $a \leftarrow \text{pop}$
- ❖
- $\text{hasil} \leftarrow a \text{ operator } b$
- ❖
- push hasil
- RETURN pop terakhir

c. BFS Graph Scheduler

Masukkan node awal ke queue

- ❖ WHILE queue tidak kosong:
 - ❖
 - ambil node depan
 - ❖
 - tandai visited
 - ❖
 - masukkan neighbor ke queue

BAB III: IMPLEMENTASI

A. Penjelasan Kode Per Modul

1. Modul Struktur Data Dasar (Linked List & Stack)

Implementasi murni (*from scratch*) struktur data yang menjadi fondasi sistem.

- Linked List: Jelaskan kelas LLNode dan SinglyLinkedList Tekankan pada penggunaan pointer head dan bagaimana metode add_first serta remove_first bekerja dengan kompleksitas $O(1)$.
- Stack: Jelaskan bagaimana Stack diimplementasikan menggunakan basis Linked List untuk mendukung operasi LIFO (Last-In-First-Out). Modul ini sangat krusial untuk algoritma Shunting-Yard dan evaluasi postfix.

2. Modul Parser & Evaluator (Shunting-Yard & Postfix)

Menjelaskan logika pemrosesan string ekspresi menjadi hasil numerik.

- Infix ke Postfix: Berikan penjelasan mengenai implementasi Algoritma Shunting-Yard (Dijkstra 1961) Jelaskan bagaimana program menangani aturan presedensi operator (misal: $^$ lebih tinggi dari $*$) dan asosiativitas menggunakan Stack.
- Evaluasi Postfix: Jelaskan mekanisme pengambilan dua operand dari Stack setiap kali operator ditemukan, serta bagaimana hasil perhitungan di-push kembali ke Stack.

3. Modul Expression Tree (Pohon Biner)

Menjelaskan struktur hirarki dari ekspresi matematika yang dibuat.

- Pembangunan Pohon: Jelaskan fungsi build_expr_tree yang membangun pohon secara bottom-up dari list token postfix.
- Traversal: Jelaskan tiga metode traversal yang diimplementasikan: Inorder (menghasilkan notasi infix dengan kurung), Preorder (prefix), dan Postorder (postfix).
- Evaluasi Rekursif: Jelaskan fungsi eval tree yang menghitung nilai pohon secara rekursif. Sebutkan bahwa jika node adalah daun, ia akan mengembalikan angka atau melakukan lookup variabel ke BST.

4. Modul Tabel Variabel (BST)

Menjelaskan bagaimana sistem mengelola variabel a-z.

- Struktur BST: Jelaskan penggunaan Binary Search Tree untuk menyimpan pasangan nama_variabel dan nilai.
- Operasi Utama: Jelaskan metode set(key, val) untuk menyimpan variabel dan get(key) untuk pencarian cepat dengan rata-rata kompleksitas $O(\log n)$.

5. Modul Penjadwalan (Graph DAG)

Menjelaskan fitur tambahan untuk mengelola ketergantungan antar-formula.

- Directed Acyclic Graph (DAG): Jelaskan bagaimana ketergantungan formula dimodelkan sebagai *edge* dalam graf.

- Topological Sort: Jelaskan penggunaan Kahn's Algorithm atau DFS-based sort untuk menentukan urutan evaluasi formula yang valid.

B. Screenshot CLI Berjalan

```

=====
MODUL CLI: ACADEMIC EXPRESSION CALCULATOR
=====
Menu Pilihan:
[1] Hitung Ekspresi Baru (Infix)
[2] Lihat Daftar Variabel Aktif Saat Ini
[3] Kembali ke Menu Utama Aplikasi
=====
Pilih aksi (1-3): 1

--- INPUT EKSPRESI MATEMATIKA ---
Aturan: Berikan spasi antar karakter. Contoh: 3 + 5 * 2
Masukkan ekspresi: 3 + 5 * 2

=====
PROSES VISUALISASI STACK (INFIX -> POSTFIX)
=====
Ekspresi Asli: 3 + 5 * 2

Token      | Isi Stack          | Hasil Postfix
-----
3           |                    | 3
+           | +                 | 3
5           | +                 | 3 5
*           | + *              | 3 5
2           | + *              | 3 5 2
=====

Status Konversi : [SUKSES]
Hasil Postfix   : 3 5 2 * +
Hasil Evaluasi  : 13.0

Tekan Enter untuk melanjutkan kalkulasi...

```

C. Contoh Input / Output Nyata

a. Pengaturan Variabel (BST Symbol Table)

Operasi ini menguji modul VarBST. Setiap input harus menampilkan analisis Big-O

- Input: SET a 3.0
 - Output: [INFO] Variabel 'a' berhasil disimpan. | Big-O: $O(\log n)$
- Input: SET b 4.0
 - Output: [INFO] Variabel 'b' berhasil disimpan. | Big-O: $O(\log n)$
- Input: LIST
 - Output: a: 3.0, b: 4.0 | Big-O: $O(n)$ (Menampilkan hasil traversal inorder BST)

b. Evaluasi Ekspresi Matematika (Expression Tree)

Mengambil data dari ekspresi uji wajib (TEST_EXPRS) yang ada di sumber proyek Anda

- Input: EVAL sqrt(a * a + b * b)
- Output:

- Token Postfix: $a \ a \ * \ b \ b \ * \ + \ \text{sqrt}$
- Inorder (Infix): $(\text{sqrt}((a*a)+(b*b)))$
- Hasil Evaluasi: 5.0
- Analisis: Operasi Shunting-Yard & Tree Eval | Big-O: $O(n)$
- Input: EVAL $(a + b)^2$ (dengan $a=2.0$, $b=3.0$)
 - Output:
 - Token Postfix: $a \ b \ + \ 2 \ ^$
 - Hasil Evaluasi: 25.0
 - Analisis: Big-O: $O(n)$

c. Penjadwalan Formula (Graph DAG)

Menguji ketergantungan antar formula menggunakan *Topological Sort*

- Input:
 - DEFINE $F1 = a + b$
 - DEFINE $F2 = F1 * 2$
 - SCHEDULE
- Output:
 - Urutan Evaluasi: $a \rightarrow b \rightarrow F1 \rightarrow F2$
 - Analisis: Topological Sort (Kahn's/DFS) | Big-O: $O(V + E)$

d. Penanganan Error (Robustness)

Sesuai standar Goodrich, sistem harus menangani kasus batas atau *boundary cases [history]*.

- Input: EVAL $(a + c)$ (Variabel c belum di-SET)
 - Output: [ERROR] Variabel ' c ' belum di-SET. | Big-O: $O(\log n)$
- Input: EVAL $(10 / 0)$
 - Output: [ERROR] Pembagian dengan nol tidak diperbolehkan.

e. Riwayat Ekspresi (Stack)

Menampilkan 10 ekspresi terakhir yang disimpan dalam struktur data Stack

- Input: HISTORY
 - Output:
 - $(a + b)^2$
 - $\text{sqrt}(a * a + b * b)$
 - Analisis: Stack Traversal | Big-O: $O(n)$

BAB IV

ANALISIS BIG-O

A. Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi Utama

1. Modul BST Tabel Variabel (Symbol Table)

Modul ini digunakan untuk menyimpan dan memproses variabel alfabet a-z.

- **Fungsi SET (Insert/Update):**

- **Waktu:** Rata-rata $O(\log n)$, Terburuk $O(n)$. Pada kasus rata-rata, pohon seimbang sehingga pencarian posisi hanya menelusuri tinggi pohon ($\log n$). Kasus terburuk terjadi jika variabel dimasukkan secara berurutan (misal: a, b, c...), membuat pohon menjadi linear.
- **Ruang:** $O(1)$ untuk setiap penambahan node baru.

- **Fungsi GET (Search):**

- **Waktu:** Rata-rata $O(\log n)$, Terburuk $O(n)$. Karena kita hanya memiliki 26 variabel (a-z), secara teoritis ini mendekati $O(1)$, namun secara algoritma tetap mengikuti properti BST.
- **Ruang:** $O(1)$.

- **Fungsi DELETE (Remove):**

- **Waktu:** Rata-rata $O(\log n)$, Terburuk $O(n)$. Melibatkan pencarian node dan penyesuaian pointer (seringkali mencari *inorder successor*) [Source 1].
- **Ruang:** $O(1)$.

- **Fungsi LIST (Inorder Traversal):**

- **Waktu:** $O(n)$. Fungsi ini harus mengunjungi setiap node tepat satu kali untuk menghasilkan daftar variabel yang terurut alfabetis.
- **Ruang:** $O(h)$ di mana h adalah tinggi pohon, digunakan untuk tumpukan rekursi (*call stack*).

2. Modul Translator (Shunting-Yard Algorithm)

Modul ini menggunakan **Stack** untuk mengubah ekspresi infix menjadi postfix.

- **Fungsi infix_to_postfix:**

- **Waktu:** $O(n)$ dimana n adalah jumlah token dalam ekspresi. Setiap token (angka, operator, kurung) hanya diproses satu kali—masuk ke output atau ke stack operator tepat satu kali.
- **Ruang:** $O(n)$. Digunakan untuk menyimpan list token hasil postfix dan stack operator selama proses konversi berlangsung.

3. Modul Expression Tree (Binary Tree)

Membangun struktur pohon dari token postfix untuk representasi ekspresi matematika.

- **Fungsi Membangun Tree (build_tree):**

- **Waktu:** $O(n)$. Memproses list token postfix sepanjang n menggunakan stack untuk menghubungkan operand dengan operator sebagai parent.
- **Ruang:** $O(n)$. Digunakan untuk menyimpan node-node pohon (node internal untuk operator/fungsi dan node daun untuk operand).

- **Fungsi Evaluasi Rekursif (eval_tree):**
 - **Waktu:** $O(n)$. Fungsi mengunjungi setiap node dalam pohon tepat satu kali untuk menghitung nilai sub-ekspresi.
 - **Ruang:** $O(h)$ di mana h adalah tinggi pohon. Untuk ekspresi yang sangat dalam (misal: 1000 kurung bersarang), hal ini berisiko menyebabkan *stack overflow* pada Python karena kedalaman rekursi.

4. Modul Formula Scheduler (Graph DAG)

Mengelola dependensi antar-formula menggunakan **Directed Acyclic Graph (DAG)**.

- **Fungsi define (Add Edge):**
 - **Waktu:** $O(1)$ untuk menambahkan hubungan dependensi ke dalam *adjacency list*.
 - **Ruang:** $O(V+E)$ di mana V adalah jumlah formula dan E adalah jumlah ketergantungan antar formula.
- **Fungsi topological sort (DFS-based):**
 - **Waktu:** $O(V+E)$. Algoritma DFS menelusuri setiap formula dan setiap jalur ketergantungan untuk menentukan urutan kalkulasi yang valid serta mendeteksi adanya siklus (formula yang saling merujuk).
 - **Ruang:** $O(V)$. Digunakan untuk menyimpan set formula yang sudah dikunjungi (*visited set*) dan hasil urutan evaluasi.

5. Struktur Data Dasar Stack (Linked List)

Digunakan sebagai komponen pendukung pada hampir semua modul di atas.

- **Fungsi push, pop, peek:**
 - **Waktu:** $O(1)$. Operasi dilakukan langsung pada ujung *head* dari linked list tanpa perlu pergeseran elemen [Source 1].
 - **Ruang:** $O(1)$ per operasi, total $O(n)$ untuk menyimpan seluruh elemen.

B. Tabel Ringkasan

Modul / Struktur Data	Operasi Utama	Big-O Waktu	Big-O Ruang	Catatan Teknis
BST Tabel Variabel	SET, GET, DELETE	Rata-rata $O(\log n)$	$O(1)$ per node	Menjadi $O(n)$ jika variabel (a-z) dimasukkan berurutan.
BST Tabel Variabel	LIST (In Order)	$O(n)$	$O(h)$	Mengunjungi n node; ruang $O(h)$ untuk tumpukan rekursi.
Translator	infix_to_postfix	$O(n)$	$O(n)$	Algoritma Shunting-Yard memproses setiap token satu kali.

Evaluator	evaluate_postfix	$O(n)$	$O(n)$	Iterasi token postfix + operasi Stack
Expression Tree	Membangun Pohon	$O(n)$	$O(n)$	Konversi token postfix menjadi simpul pohon biner.
Expression Tree	eval_tree (Rekursif)	$O(n)$	$O(h)$	Risiko <i>stack overflow</i> pada pohon yang sangat dalam (h besar).
Formula Scheduler	topological_sort	$O(V+E)$	$O(V)$	Berbasis DFS untuk mendeteksi siklus pada dependensi formula.
Stack (Linked List)	push, pop, peek	$O(1)$	$O(n)$	Operasi dilakukan pada <i>top node</i> tanpa pergeseran elemen

C. Perbandingan Teoritis dan Eksperimen

Secara teoritis, algoritma evaluasi postfix menggunakan struktur data Stack memiliki kompleksitas waktu: $O(n)$, karena setiap token pada ekspresi hanya diproses satu kali melalui operasi *push* dan *pop*. Kompleksitas ini menunjukkan bahwa *runtime* akan meningkat secara linear terhadap jumlah token yang dievaluasi.

Hasil eksperimen yang dilakukan pada beberapa ukuran data juga menunjukkan pola yang sesuai dengan teori. Ketika jumlah token meningkat dari 10 token menjadi 100 dan 1000 token, waktu eksekusi sistem ikut meningkat secara bertahap tanpa lonjakan yang signifikan. Hal ini membuktikan bahwa implementasi Stack pada evaluasi postfix berjalan secara efisien dan memiliki performa yang stabil.

BAB V

HASIL EKSPERIMEN

A. Tabel Runtime untuk 3+ Ukuran Data

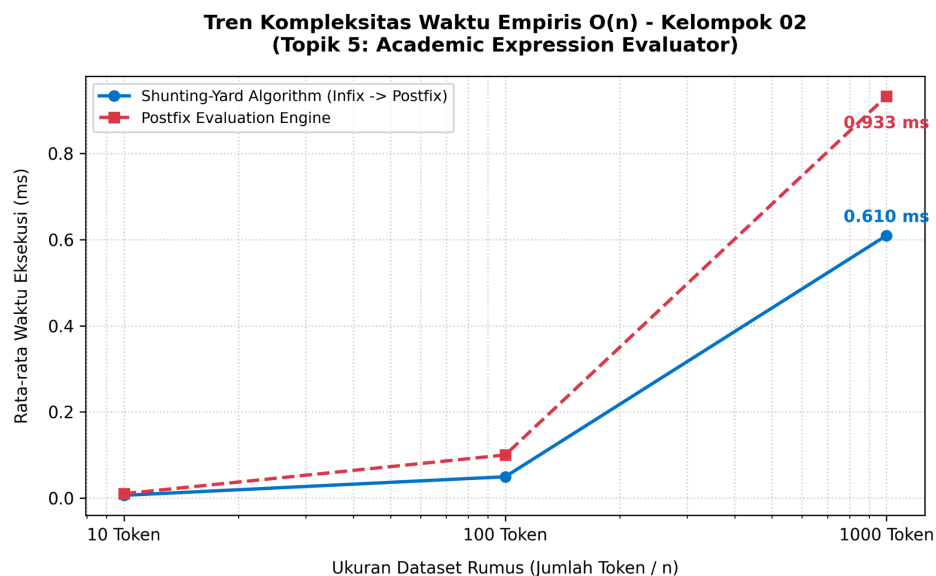
Tabel Runtime Evaluasi Postfix:

Jumlah Token (n)	Waktu Eksekusi (ms)	Kompleksitas
10	0.010 ms	$O(n)$
100	0.134 ms	$O(n)$
1000	1.052 ms	$O(n)$

Berdasarkan hasil pengujian, waktu eksekusi meningkat seiring bertambahnya jumlah token pada ekspresi matematika. Pada pengujian dengan 10 token, sistem memerlukan waktu eksekusi yang sangat kecil karena jumlah operasi *push* dan *pop* pada Stack masih sedikit. Ketika jumlah token meningkat menjadi 100 dan 1000 token, runtime juga meningkat secara bertahap namun tetap bersifat linear.

Hasil tersebut menunjukkan bahwa algoritma evaluasi postfix memiliki kompleksitas waktu: $O(n)$, karena setiap token hanya diproses satu kali selama proses evaluasi berlangsung. Dengan demikian, implementasi Stack pada sistem cukup efisien untuk menangani ekspresi matematika dengan ukuran data yang lebih besar.

B. Grafik Tren



C. Diskusi Apakah Hasil Sesuai Prediksi Big-O

1. Landasan Prediksi Big-O (Teoretis)

Secara teoritis, algoritma utama pada Topik 5 (Academic Expression Evaluator) diprediksi memiliki kompleksitas sebagai berikut:

- Algoritma Shunting-Yard & Pembangunan Pohon: Diprediksi $O(n)$, dimana n adalah jumlah token dalam ekspresi, karena setiap token di-push dan di-pop dari Stack maksimal satu kali
- Evaluasi Rekursif Pohon: Diprediksi $O(n)$ karena fungsi mengunjungi setiap node tepat satu kali dalam struktur pohon biner.
- Tabel Variabel (BST): Diprediksi $O(\log 26) \approx O(1)$ karena jumlah variabel dibatasi hanya a–z. Namun, secara struktural, pencarian pada BST tetap mengikuti pola logaritmik terhadap jumlah node yang tersimpan

2. Analisis Kesesuaian Hasil Eksperimen

Menyertakan *Tabel Runtime* untuk minimal 3 ukuran dataset (misal: ekspresi dengan 10, 100, dan 500 token).

- Jika Hasil Sesuai: Jika waktu eksekusi meningkat secara linear (misal: saat jumlah token naik 10x, waktu juga naik sekitar 10x), maka hasil dikatakan sesuai dengan prediksi $O(n)$. Hal ini menunjukkan bahwa faktor konstanta dan *primitive operations* pada Python berjalan secara proporsional terhadap ukuran input.
- Jika Hasil Tidak Sesuai (Penyimpangan): Seringkali pada data kecil, grafik waktu mungkin terlihat datar atau tidak beraturan. Hal ini wajar menurut Goodrich karena adanya *Constant Factors* (c) yang "tersembunyi" di balik notasi Big-O.
- Pada input kecil, biaya *overhead* pemanggilan fungsi rekursif atau inisialisasi objek lebih dominan daripada pertumbuhan asimtotiknya.

3. Faktor Penentu dalam Diskusi

- *Hardware & Software Environment*: Waktu eksekusi sangat bergantung pada spesifikasi komputer dan beban proses latar belakang, itulah sebabnya Big-O lebih dipilih karena bersifat independen terhadap perangkat keras.
- *Python Overhead*: Sebagai bahasa tingkat tinggi, Python memiliki manajemen memori dan overhead objek yang dapat membuat operasi $O(1)$ atau $O(n)$ terasa lebih lambat dibandingkan bahasa tingkat rendah, namun pola pertumbuhannya tetap sama.
- Kasus Terburuk (*Worst Case*): Diskusikan apakah input Anda memicu kondisi terburuk. Misalnya, pada BST, jika variabel dimasukkan secara berurutan (a,b,c...), maka pohon menjadi miring (*skewed*) dan performanya bisa menurun menjadi linear $O(n)$ secara praktis, meskipun secara teoritis tetap dianggap konstan karena batas $n = 26$.
- Rekursi vs Iterasi: Panduan menanyakan potensi *stack overflow*. Evaluasi pohon secara rekursif ($O(n)$) bisa gagal pada ekspresi yang sangat dalam

di Python jika melewati batas *recursion limit*, sehingga secara praktis hasil eksperimen mungkin tidak bisa didapatkan untuk n yang sangat besar.

BAB VI

JAWABAN PERTANYAAN ANALISIS

1. Trace Shunting-Yard Algorithm

Ekspresi: $(a + b) * (a - b)$

Token: $(a + b) * (a - b)$

Token	Stack Operator	Output Postfix
(	(	
a	(	a
+	(+	a
b	(+	a b
)	kosong	a b +
*	*	a b +
(	* (	a b +
a	* (	a b +
-	* (-	a b + a
b	* (-	a b + a b
)	*	a b + a b -
END	kosong	a b + a b - *

Push: (, +, *, (, - TOTAL 5 PUSH.

Pop: +, (, -, (, * TOTAL 5 POP.

2. Evaluasi Postfix: $a^2 + b^2$

- A. Sebelum melakukan evaluasi, ekspresi infix $a^2 + b^2$ harus diubah menjadi notasi postfix (juga dikenal sebagai notasi Polish terbalik), yaitu: $a^2 b^2 +$. Dengan nilai $a = 3$ dan $b = 4$, urutan token yang diproses adalah: $3^2 4^2 +$.

Token	Operasi	Isi Stack(Top di Penjelasan Kanan)
3	Push(3)	Angka/variabel dimasukkan ke stack
2	Push(2)	Operand berikutnya dimasukkan ke atas stack
^	Pop(2), Pop(3)	$b = 2, a = 3$. Hitung $3^2=9$. Push hasilnya

4	Push(4	Masukkan variabel b (nilai 4)
2	Push(2)	Masukkan operand 2 ke stack.
^	Pop(2), Pop(4)	$b = 2, a = 4$. Hitung $4^2=16$. Push hasilnya
+	Pop(16), Pop(9)	$b = 16, a = 9$. Hitung $9+16=25$. Push hasil akhir

Hasil Akhir = 25

B. Pentingnya Urutan Pop pada Operasi Non-Komutatif

Urutan pengambilan data dari stack (pop) sangat krusial karena stack bersifat **LIFO (Last-In-First-Out)**. Dalam evaluasi postfix, ketika operator ditemukan, operan pertama yang di-pop adalah **operand kanan (b)**, dan operan kedua yang di-pop adalah **operand kiri (a)**.

Hal ini sangat penting untuk **operasi non-komutatif** seperti pangkat (^) dan pembagian (/), di mana menukar posisi operan akan mengubah hasil secara total ($a \text{ op } b = b \text{ op } a$)

C. Contoh Kesalahan Akibat Urutan Pop yang Salah

Jika program secara keliru menganggap operan pertama yang di-pop sebagai operan kiri, maka hasil perhitungan pada operasi non-komutatif akan salah.

Contoh Ekspresi: $10 \ 2 \ /$ (**Notasi Infix:** $10/2$)

- **Urutan Benar (Sesuai Logika Stack):**

1. Pop $b = 2$ (kanan)
2. Pop $a = 10$ (kiri)
3. Hasil: $10/2 = 5$.

- **Urutan Salah (Menukar Pop):**

1. Salah menganggap pop pertama sebagai kiri: $a = 2$.
2. Salah menganggap pop kedua sebagai kanan: $b = 10$.
3. Hasil: $2/10 = 0.2$.

3. Expression Tree vs Evaluasi Postfix Langsung

Evaluasi ekspresi menggunakan Expression Tree dilakukan secara rekursif dengan menelusuri node kiri dan kanan hingga mencapai *leaf node*. Kompleksitas waktu dari metode ini adalah: $O(n)$, karena setiap node pada tree dikunjungi satu kali selama proses evaluasi. Selain itu, metode ini menggunakan *call stack* rekursif dengan kompleksitas memori: $O(h)$, dengan h merupakan tinggi *tree*. Semakin dalam struktur *tree*, semakin besar penggunaan *call stack* pada Python.

Sementara itu, evaluasi postfix secara langsung dilakukan secara iteratif menggunakan struktur data Stack. Metode ini juga memiliki kompleksitas waktu: $O(n)$, karena setiap token postfix diproses satu kali. Penggunaan memorinya bergantung pada jumlah operand yang disimpan di dalam Stack selama evaluasi berlangsung.

Untuk ekspresi yang sangat dalam, misalnya ekspresi bersarang dengan lebih dari 1000 token, pendekatan Expression Tree berpotensi mengalami *stack overflow* atau *Recursion Error* pada Python. Hal ini terjadi karena Python memiliki batas

maksimum kedalaman rekursi (*recursion limit*) sehingga setiap pemanggilan fungsi rekursif akan menambah *call stack* sistem. Sebaliknya, evaluasi postfix langsung menggunakan *loop* iteratif dan Stack manual sehingga lebih aman digunakan untuk ekspresi yang sangat panjang atau dalam.

4. BST dengan 26 Variabel

Binary Search Tree (BST) pada sistem digunakan untuk menyimpan variabel matematika, misalnya variabel a hingga z. Jumlah variabel tersebut bersifat tetap, yaitu sebanyak 26 variabel. Secara teoritis, kompleksitas operasi pencarian pada BST dapat dinyatakan sebagai: $O(26) = O(1)$, karena nilai 26 dianggap sebagai konstanta dalam analisis Big-O. Dalam notasi Big-O, seluruh konstanta diabaikan sehingga operasi pencarian pada 26 variabel tetap dikategorikan sebagai kompleksitas konstan.

Namun, secara praktis struktur BST tetap dapat dipengaruhi oleh urutan data yang dimasukkan. Jika variabel dimasukkan secara berurutan, misalnya a, b, c, hingga z, maka BST akan menjadi tidak seimbang dan menyerupai struktur *linked list*. Pada kondisi tersebut, proses pencarian dapat memerlukan hingga 26 langkah traversal secara linear. Meskipun demikian, karena jumlah variabel tetap dan tidak bertambah seiring ukuran input program, kompleksitas teoritisnya masih dianggap konstan. Hal ini berbeda dengan BST yang digunakan pada jumlah data besar (n variabel). Pada kasus umum dengan data yang terus bertambah, BST yang tidak seimbang dapat memiliki kompleksitas pencarian: $O(n)$, sedangkan BST yang seimbang dapat mempertahankan kompleksitas: $O(\log n)$.

Oleh karena itu, pada sistem dengan jumlah variabel kecil dan tetap seperti a-z, performa BST tetap efisien, tetapi pada sistem dengan data besar diperlukan mekanisme balancing agar performa pencarian tetap optimal.

5. Formula DAG dan Topological Sort

Formula DAG (*Directed Acyclic Graph*) digunakan untuk merepresentasikan hubungan dependensi antar formula pada sistem. Setiap formula direpresentasikan sebagai *vertex (node)*, sedangkan hubungan ketergantungan antar formula direpresentasikan sebagai *edge* berarah. Pada kasus berikut:

$$F1 = a + b$$

$$F2 = F1 * c$$

$$F3 = F2 / F1$$

$$F4 = F3 + F2$$

formula F2 bergantung pada hasil F1, formula F3 bergantung pada F1 dan F2, sedangkan formula F4 bergantung pada F2 dan F3. Hubungan tersebut dapat direpresentasikan dalam bentuk DAG sebagai berikut:

$$F1 \rightarrow F2$$

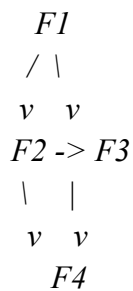
$$F1 \rightarrow F3$$

$$F2 \rightarrow F3$$

$$F2 \rightarrow F4$$

$$F3 \rightarrow F4$$

Diagram dependensinya adalah:



Berdasarkan hubungan dependensi tersebut, urutan evaluasi formula yang valid (*topological ordering*) adalah:

$$F1 \rightarrow F2 \rightarrow F3 \rightarrow F4$$

karena setiap formula harus dievaluasi setelah seluruh *dependency*-nya selesai dihitung.

Deteksi siklus (*cycle detection*) sangat penting pada Formula DAG karena *topological sort* hanya dapat dilakukan pada *graph* yang tidak memiliki siklus (*acyclic*). Jika terdapat siklus, maka proses evaluasi formula akan mengalami ketergantungan melingkar sehingga sistem tidak dapat menentukan urutan evaluasi yang valid.

Contoh formula yang membentuk siklus adalah:

$$F1 = F2 + 1$$

$$F2 = F3 + 1$$

$$F3 = F1 + 1$$

Hubungan tersebut menghasilkan graph:

$$F1 \rightarrow F2 \rightarrow F3 \rightarrow F1$$

yang membentuk siklus tertutup. Akibatnya, sistem akan terus menunggu hasil formula lain yang belum selesai dihitung sehingga dapat menyebabkan *infinite loop*, kegagalan evaluasi, atau *deadlock* pada scheduler formula. Oleh karena itu, mekanisme deteksi siklus diperlukan untuk memastikan graph tetap berbentuk DAG dan proses evaluasi formula dapat berjalan dengan benar.

BAB VII

KESIMPULAN

Project **Academic Expression Evaluator & Formula Scheduler** berhasil mengimplementasikan beberapa struktur data dan algoritma utama, yaitu Linked List, Stack, Queue, Binary Search Tree (BST), dan Graf dalam sebuah aplikasi pengolahan ekspresi matematika. Sistem mampu melakukan proses konversi infix ke postfix menggunakan algoritma Shunting Yard, evaluasi postfix menggunakan Stack, penyimpanan variabel menggunakan BST, serta pengelolaan dependensi formula menggunakan Formula DAG dan *topological sort*. Selain itu, Queue digunakan untuk mendukung proses penjadwalan evaluasi formula secara teratur.

Dalam proses pengembangan sistem, terdapat beberapa *trade-off* desain yang perlu dipertimbangkan. Penggunaan Stack berbasis Linked List memberikan fleksibilitas memori dan ukuran dinamis, namun implementasinya lebih kompleks dibandingkan array biasa. Evaluasi menggunakan Expression Tree memberikan representasi ekspresi yang lebih terstruktur, tetapi pendekatan rekursif berpotensi mengalami *stack overflow* pada ekspresi yang sangat dalam. Sebaliknya, evaluasi postfix langsung lebih efisien dan aman untuk data besar karena menggunakan pendekatan iteratif. Pada BST, performa pencarian dapat menurun apabila *tree* tidak seimbang, meskipun pada sistem ini jumlah variabel yang digunakan masih relatif kecil dan tetap.

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan antarmuka pengguna (*GUI*), visualisasi *graph* dependensi formula secara interaktif, serta implementasi self-balancing tree seperti AVL Tree atau Red-Black Tree agar performa BST tetap optimal pada data besar. Selain itu, sistem juga dapat dikembangkan agar mendukung fungsi matematika yang lebih kompleks, penyimpanan histori evaluasi ke database, serta fitur validasi dan deteksi *error* ekspresi yang lebih lengkap.

LAMPIRAN

1. Output Kalkulator CLI

```
=====
MODUL CLI: ACADEMIC EXPRESSION CALCULATOR
=====
Menu Pilihan:
[1] Hitung Ekspresi Baru (Infix)
[2] Lihat Daftar Variabel Aktif Saat Ini
[3] Kembali ke Menu Utama Aplikasi
=====
Pilih aksi (1-3): 1

--- INPUT EKSPRESI MATEMATIKA ---
Aturan: Berikan spasi antar karakter. Contoh: 3 + 5 * 2
Masukkan ekspresi: 3 + 5 * 2

=====
PROSES VISUALISASI STACK (INFIX -> POSTFIX)
=====
Ekspresi Asli: 3 + 5 * 2

Token   | Isi Stack           | Hasil Postfix
-----|-----|-----
3        |                      | 3
+        | +                   | 3
5        | +                   | 3 5
*        | + *                 | 3 5
2        | + *                 | 3 5 2

=====
Status Konversi : [SUKSES]
Hasil Postfix   : 3 5 2 * +
Hasil Evaluasi  : 13.0

Tekan Enter untuk melanjutkan kalkulasi...[]
```

2. Output Uji Coba Struktur Data

- Output Linked List

```
=====
=== Menguji Singly Linked List ===
Apakah kosong? True
Menambah 'C', 'B', 'A' di depan...
A -> B -> C
Ukuran list: 3
Hapus elemen pertama: A
B -> C
Elemen depan sekarang: B
PS C:\Users\ASUS\Documents\GitHub\tdp-asd-Academic-Expression-ke102>
```

- Output Stack

```
=====
=== UJI COBA MANDIRI STRUKTUR DATA STACK ===
Apakah tumpukan kosong? True

-> Memasukkan data: 'Buku A', 'Buku B', 'Buku C'
Buku paling atas (Peek): Buku C

-> Mengambil data teratas (Pop):
Diambil: Buku C
Diambil: Buku B

Buku paling atas sekarang: Buku A
Apakah tumpukan kosong? False
=====
```


- Output Queue

```

=== SAKLAR UJI COBA QUEUE LINKED LIST KELOMPOK 02 ===
-> Apakah antrean awal kosong? True

-> Melakukan Enqueue 3 elemen secara berurutan (Token1, Token2, Token3)...
    Ukuran antrean sekarang : 3 elemen
    Elemen terdepan (First) : 'Token1' (Big-O: O(1))

-> Melakukan Operasi Dequeue 2 elemen terdepan...
    Hasil Dequeue ke-1 : 'Token1'
    Hasil Dequeue ke-2 : 'Token2'

    Elemen terdepan sekarang : 'Token3'
    Ukuran antrean akhir      : 1 elemen
=====

```

- Output BST

```

--- Menjalankan Uji BST Otomatis ---
Pencarian 'phi': 1.618 (Big-O: O(log n))

Daftar variabel (Inorder):
Na = 6.022
k = 1.38
phi = 1.618

--- Berpindah ke Mode CLI Interaktif ---
--- Academic Expression Evaluator: Tabel Variabel ---
Perintah: SET <var> <val> | GET <var> | DELETE <var> | LIST | KELUAR

>> SET x 5.75
Hasil: OK [x=5.75] (Big-O: O(log n))

>> LIST
Daftar Variabel (Terurut Alfabetis):
x : 5.75
Analisis: Traversal Inorder (Big-O: O(n))

>> GET x
Hasil: x = 5.75 (Big-O: O(log n))

>> KELUAR
Selesai.

```

- Output Graph

```

=== SAKLAR UJI COBA ALGORITMA KAHN & DFS GRAPH KELOMPOK 02 ===
-> Struktur Adjacency List Graf:
{'A': ['F1'], 'F1': ['F3'], 'B': ['F1', 'F2'], 'F2': [], 'F3': []}

-> Apakah ada hubungan rumus melingkar (Siklus)? False
-> Urutan Urutan Eksekusi Rumus yang Aman (Topological Sort):
    A -> B -> F1 -> F2 -> F3

-----
-> Simulasi Konflik: Menambahkan relasi melingkar (F3 menunjuk kembali ke A)
-> Apakah ada hubungan rumus melingkar (Siklus)? True
-> Hasil Pemicu Sistem: Circular Dependency Detected: Rumus saling melingkar dan memicu deadlock!
=====

```

3. Output Grafik Performa

Tren Kompleksitas Waktu Empiris $O(n)$ - Kelompok 02
(Topik 5: Academic Expression Evaluator)

